

```
import os
os.environ["OPENAI_API_KEY"] = " "
```

```
!pip install -q langchain-openai langchain-core requests
```

```
===== 62.4/62.4 kB 2.0 MB/s eta 0:00:00
===== 1.2/1.2 MB 28.0 MB/s eta 0:00:00
```

```
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
import requests
```

```
# tool create

@tool
def multiply(a: int, b: int) -> int:
    """Given 2 numbers a and b this tool returns their product"""
    return a * b
```

```
print(multiply.invoke({'a':3, 'b':4}))
```

```
12
```

```
multiply.name
```

```
'multiply'
```

```
multiply.description
```

```
'Given 2 numbers a and b this tool returns their product'
```

```
multiply.args
```

```
{'a': {'title': 'A', 'type': 'integer'},
 'b': {'title': 'B', 'type': 'integer'}}
```

```
llm = ChatOpenAI()
```

```
l = llm.bind_tools([multiply]) # tool binding this is
```

```
l.invoke('Hi how are you')
```

```
AIMessage(content="Hello! I'm here and ready to assist you. How can I help
you today?", additional_kwargs={'refusal': None}, response_metadata=
{'token_usage': {'completion_tokens': 20, 'prompt_tokens': 58,
'total_tokens': 78, 'completion_tokens_details':
{'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0,
'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens':
0, 'cached_tokens': 0}}, 'model_name': 'gpt-3.5-turbo-0125',
'system_fingerprint': None, 'id': 'chatcmpl-BR8rNpETFpqE6bIK5rNVUiydjkzv4',
'finish_reason': 'stop', 'logprobs': None}, id='run-325a9e76-6f20-4c7b-93ac-
4740de7b21da-0', usage_metadata={'input_tokens': 58, 'output_tokens': 20,
```

```
'total_tokens': 78, 'input_token_details': {'audio': 0, 'cache_read': 0},  
'output_token_details': {'audio': 0, 'reasoning': 0}})
```

```
query = HumanMessage('can you multiply 3 with 1000')
```

```
messages = [query]
```

```
messages
```

```
[HumanMessage(content='can you multiply 3 with 1000', additional_kwargs={},  
response_metadata={})]
```

```
result = l.invoke(messages)
```

```
messages.append(result)
```

```
messages
```

```
[HumanMessage(content='can you multiply 3 with 1000', additional_kwargs={},  
response_metadata={}),  
AIMessage(content='', additional_kwargs={'tool_calls': [{'id':  
'call_RxxH1pPDylDECUwpRe7MXkJi', 'function': {'arguments':  
{'a': 3, 'b': 1000}}, 'name': 'multiply', 'type': 'function'}]}, 'refusal':  
None}, response_metadata={'token_usage': {'completion_tokens': 19,  
'prompt_tokens': 63, 'total_tokens': 82, 'completion_tokens_details':  
{'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0,  
'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens':  
0, 'cached_tokens': 0}}, 'model_name': 'gpt-3.5-turbo-0125',  
'system_fingerprint': None, 'id': 'chatcmpl-BR8rS3DNc8cckcVLJmBDxHENKULV',  
'finish_reason': 'tool_calls', 'logprobs': None}, id='run-8035ac83-7820-4681-  
b8c0-1d15aa24ca77-0', tool_calls=[{'name': 'multiply', 'args': {'a': 3, 'b':  
1000}, 'id': 'call_RxxH1pPDylDECUwpRe7MXkJi', 'type': 'tool_call'}],  
usage_metadata={'input_tokens': 63, 'output_tokens': 19, 'total_tokens': 82,  
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details':  
{'audio': 0, 'reasoning': 0}})]
```

```
tool_result = multiply.invoke(result.tool_calls[0])
```

```
tool_result
```

```
ToolMessage(content='3000', name='multiply',  
tool_call_id='call_RxxH1pPDylDECUwpRe7MXkJi')
```

```
messages.append(tool_result)
```

```
messages
```

```
[HumanMessage(content='can you multiply 3 with 1000', additional_kwargs={},  
response_metadata={}),  
AIMessage(content='', additional_kwargs={'tool_calls': [{'id':  
'call_RxxH1pPDylDECUwpRe7MXkJi', 'function': {'arguments':  
{'a': 3, 'b': 1000}}, 'name': 'multiply', 'type': 'function'}]}, 'refusal':  
None}, response_metadata={'token_usage': {'completion_tokens': 19,  
'prompt_tokens': 63, 'total_tokens': 82, 'completion_tokens_details':  
{'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0,  
'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens':
```

```
0, 'cached_tokens': 0}}, 'model_name': 'gpt-3.5-turbo-0125',  
'system_fingerprint': None, 'id': 'chatcmpl-BR8rS3DNc8cckcVLJMmBDxHENKULV',  
'finish_reason': 'tool_calls', 'logprobs': None}, id='run-8035ac83-7820-4681-  
b8c0-1d15aa24ca77-0', tool_calls=[{'name': 'multiply', 'args': {'a': 3, 'b':  
1000}, 'id': 'call_RxxH1pPDylDECUwpRe7MXkji', 'type': 'tool_call'}],  
usage_metadata={'input_tokens': 63, 'output_tokens': 19, 'total_tokens': 82,  
'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details':  
{'audio': 0, 'reasoning': 0}}),  
ToolMessage(content='3000', name='multiply',  
tool_call_id='call_RxxH1pPDylDECUwpRe7MXkji')]
```

```
l.invoke(messages).content
```

```
'The product of 3 and 1000 is 3000.'
```

```
# tool create  
from langchain_core.tools import InjectedToolArg  
from typing import Annotated  
  
@tool  
def get_conversion_factor(base_currency: str, target_currency: str) -> float:  
    """  
    This function fetches the currency conversion factor between a given base c  
    """  
    url = f'https://v6.exchangerate-api.com/v6/c754eab14ffab33112e380ca/pair/{t  
    response = requests.get(url)  
  
    return response.json()  
  
@tool  
def convert(base_currency_value: int, conversion_rate: Annotated[float, Injec  
    """  
    given a currency conversion rate this function calculates the target curren  
    """  
  
    return base_currency_value * conversion_rate
```

```
convert.args
```

```
{'base_currency_value': {'title': 'Base Currency Value', 'type': 'integer'},  
'conversion_rate': {'title': 'Conversion Rate', 'type': 'number'}}
```

```
get_conversion_factor.invoke({'base_currency': 'USD', 'target_currency': 'INR'})
```

```
{'result': 'success',  
'documentation': 'https://www.exchangerate-api.com/docs',  
'terms_of_use': 'https://www.exchangerate-api.com/terms',  
'time_last_update_unix': 1745798401,  
'time_last_update_utc': 'Mon, 28 Apr 2025 00:00:01 +0000',  
'time_next_update_unix': 1745884801,  
'time_next_update_utc': 'Tue, 29 Apr 2025 00:00:01 +0000',  
'base_code': 'USD',  
'target_code': 'INR',  
'conversion_rate': 85.4741}
```

```
convert.invoke({'base_currency_value': 10, 'conversion_rate': 85.16})
```

```
851.5999999999999
```

```
llm = ChatOpenAI()
```

```
llm_with_tools = llm.bind_tools([get_conversion_factor, convert]) # tool binding
```

```
messages = [HumanMessage('What is the conversion factor between INR and USD,
```

```
messages
```

```
[HumanMessage(content='What is the conversion factor between INR and USD, and  
based on that can you convert 10 inr to usd', additional_kwargs={},  
response_metadata={})]
```

```
ai_message = llm_with_tools.invoke(messages)
```

```
messages.append(ai_message)
```

```
ai_message.tool_calls
```

```
[{'name': 'get_conversion_factor',  
  'args': {'base_currency': 'INR', 'target_currency': 'USD'},  
  'id': 'call_PKL8v7zwmphzNel0MnvjjGvY',  
  'type': 'tool_call'},  
 {'name': 'convert',  
  'args': {'base_currency_value': 10},  
  'id': 'call_vRdld30yHTKFGcTQPumgzH5u',  
  'type': 'tool_call'}]
```

```
import json
```

```
for tool_call in ai_message.tool_calls:  
    # execute the 1st tool and get the value of conversion rate  
    if tool_call['name'] == 'get_conversion_factor':  
        tool_message1 = get_conversion_factor.invoke(tool_call)  
        # fetch this conversion rate  
        conversion_rate = json.loads(tool_message1.content)['conversion_rate']  
        # append this tool message to messages list  
        messages.append(tool_message1)  
    # execute the 2nd tool using the conversion rate from tool 1  
    if tool_call['name'] == 'convert':  
        # fetch the current arg  
        tool_call['args']['conversion_rate'] = conversion_rate  
        tool_message2 = convert.invoke(tool_call)  
        messages.append(tool_message2)
```

```
messages
```

```
[HumanMessage(content='What is the conversion factor between INR and USD, and  
based on that can you convert 10 inr to usd', additional_kwargs={},  
response_metadata={}),  
 AIMessage(content='', additional_kwargs={'tool_calls': [{'id':  
'call_PKL8v7zwmphzNel0MnvjjGvY', 'function': {'arguments': '{"base_currency":
```

```
"INR", "target_currency": "USD"}', 'name': 'get_conversion_factor'}, 'type':
'function'}, {'id': 'call_vRdld30yHTKFGcTQPumgzH5u', 'function':
{'arguments': {'"base_currency_value": 10}', 'name': 'convert'}, 'type':
'function'}], 'refusal': None}, response_metadata={'token_usage':
{'completion_tokens': 53, 'prompt_tokens': 123, 'total_tokens': 176,
'completion_tokens_details': {'accepted_prediction_tokens': 0,
'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0},
'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}},
'model_name': 'gpt-3.5-turbo-0125', 'system_fingerprint': None, 'id':
'chatcmpl-BR8rovupYuNnIbG041T003knQ2Cr9', 'finish_reason': 'tool_calls',
'logprobs': None}, id='run-5ee0506a-8be8-4d32-8f30-f939d59b11a8-0',
tool_calls=[{'name': 'get_conversion_factor', 'args': {'base_currency':
'INR', 'target_currency': 'USD'}, 'id': 'call_PKL8v7zwmphzNel0MnvjjGvY',
'type': 'tool_call'}, {'name': 'convert', 'args': {'base_currency_value': 10,
'conversion_rate': 0.0117}, 'id': 'call_vRdld30yHTKFGcTQPumgzH5u', 'type':
'tool_call'}], usage_metadata={'input_tokens': 123, 'output_tokens': 53,
'total_tokens': 176, 'input_token_details': {'audio': 0, 'cache_read': 0},
'output_token_details': {'audio': 0, 'reasoning': 0}}),
ToolMessage(content='{"result": "success", "documentation":
https://www.exchangerate-api.com/docs, "terms_of_use":
https://www.exchangerate-api.com/terms, "time_last_update_unix":
1745798401, "time_last_update_utc": "Mon, 28 Apr 2025 00:00:01 +0000",
"time_next_update_unix": 1745884801, "time_next_update_utc": "Tue, 29 Apr
2025 00:00:01 +0000", "base_code": "INR", "target_code": "USD",
"conversion_rate": 0.0117}', name='get_conversion_factor',
tool_call_id='call_PKL8v7zwmphzNel0MnvjjGvY'),
ToolMessage(content='0.117', name='convert',
tool_call_id='call_vRdld30yHTKFGcTQPumgzH5u'))]
```

```
llm_with_tools.invoke(messages).content
```

```
'The conversion factor between INR and USD is 0.0117. \n\nBased on this conve
rsion factor, 10 INR is equal to 0.117 USD.'
```

```
from langchain.agents import initialize_agent, AgentType

# Step 5: Initialize the Agent ---
agent_executor = initialize_agent(
    tools=[get_conversion_factor, convert],
    llm=llm,
    agent=AgentType.STRUCTURED_CHAT_ZERO_SHOT_REACT_DESCRIPTION, # using ReA
    verbose=True # shows internal thinking
)
```

```
# --- Step 6: Run the Agent ---
user_query = "Hi how are you?"

response = agent_executor.invoke({"input": user_query})
```

```
> Entering new AgentExecutor chain...
I'm here and ready to help! How can I assist you today?

> Finished chain.
```

Start coding or [generate](#) with AI.

